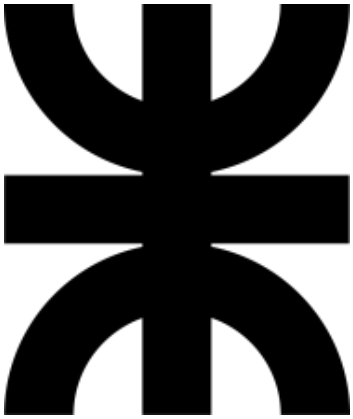
Universidad Tecnológica Nacional
Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ingeniería y Calidad de Software



TEMA: TDD - Test Driven Development

Año: 2026

Docentes

- Adjunto: Ing. Laura Covaro
- Auxiliares de Trabajos Prácticos
    - Ing. Cecilia Massano
    - Ing. Constanza Garnero
- Adscripto: Ezequiel Izaguirre

Curso: 4K3

Grupo Nro: 7

Integrantes:

- 95028 Alvaretto Caterina
- 94309 Arce Sayago Camila
- 431124 Cardozo Soledad Sofía
- 84037 Ficarra Carla Valentina
- 88440 Garcia Pintos Valentin
- 78797 Ostermann Guillermo
- 94017 Pereyra Agustín
- 97376 Pregot Ulises Valentin
- 86363 Pucheta Lanfranco
- 90112 Terreno Monla Florencia Sofia
- 92380 Vaca Sofia Eugenia

Índice

Índice **2**
Consignas **3**
Desarrollo **4**
User Story 4
Reglas de Negocio 4
Diseño de la Solución 5
Pruebas Unitarias 6
Conclusión 10

Consignas

Implementar una funcionalidad determinada utilizando el enfoque de Desarrollo Conducido por Pruebas (TDD), en el lenguaje de programación elegido por el grupo, utilizando el enfoque Red-Green-Refactor para construir el código desde las pruebas.

Objetivo: Aplicar el enfoque TDD para diseñar y construir software a partir de pruebas automatizadas, poniendo en práctica buenas prácticas de diseño y desarrollo orientado a la calidad desde el inicio.

Propósito: Familiarizarse con el ciclo TDD (Red-Green-Refactor), incorporando buenas prácticas de diseño y testing, y fortaleciendo la disciplina del desarrollo guiado por pruebas como base para un código de calidad.

Entradas:

- Conceptos teóricos sobre el tema desarrollados en clase.
- Bibliografía referenciada sobre el tema.
- Enunciado del Trabajo Práctico 2: EcoHarmony Park
- Definición completa de las User Stories correspondientes al Trabajo Práctico 2 “Requerimientos Ágiles – User Stories y Estimaciones”
- Reglas de estilo de código seleccionadas por el grupo para el Trabajo Práctico 2

Salida:

- Implementación funcional de la User Story asignada
- Conjunto de pruebas automatizadas que cubren los distintos casos definidos.
- Documento que justifique las decisiones de diseño tomadas.

Instrucciones:

- Seleccionar una User Story a implementar de entre las siguientes opciones:
    - Comprar entradas (grupos impares)
    - Inscribirme a actividad (grupos pares)
- Utilizando el lenguaje y documento de estilos de código seleccionado para el Trabajo Práctico 2 “Requerimientos Ágiles – User Stories y Estimaciones”, implementar la funcionalidad aplicando el ciclo TDD:
    - Red: Escribir una prueba que falle.
    - Green: Implementar el código mínimo para que pase.
    - Refactor: Mejorar el diseño del código sin romper las pruebas

Desarrollo

User Story

Comprar Entrada **COMO visitante QUIERO comprar una entrada PARA asegurar mi visita al parque.**	**8**
Criterios de Aceptación: ● Debe indicar la fecha de visita deseada, la cantidad de entradas requeridas, la edad de cada ● visitante y tipo de pase (VIP o regular). ● La fecha de visita guiada puede ser del día actual o futuro. ● Debe enviar un mensaje de confirmación vía mail ● Debe redirigir a mercado pago al confirmar la compra si el pago es con tarjeta de crédito ● La fecha de la visita debe estar dentro de los días en que el parque está abierto. ● Debe seleccionar la forma de pago: efectivo en caso de querer pagar en boletería o con tarjeta ● La cantidad de entradas requeridas no debe ser mayor a 10 ● Al finalizar la compra se debe informar la cantidad de entradas compradas y la fecha. ● Se debe permitir la compra de entradas solo a usuarios registrados.	
Pruebas de usuario: ● Probar comprar una entrada indicando la fecha de visita dentro de los días disponibles, una ● cantidad de entradas menor a 10, la edad de todos los visitantes, el tipo de pase, la forma de ● pago con tarjeta mediante Mercado pago y recepción del mail de confirmación (pasa) ● Probar comprar entradas sin seleccionar forma de pago (falla) ● Probar comprar entradas ingresando una fecha de visita en la cual el parque se encuentra ● cerrado (falla) ● Probar comprar entradas ingresando una cantidad de entradas mayor a 10 (falla)	

Reglas de Negocio

Además de los criterios de aceptación, la implementación aplica las siguientes reglas del enunciado de Eco Harmony Park:

- El parque abre de martes a domingo, de 8:30 a 19:00, y cierra los lunes y los feriados (25 de diciembre y 1 de enero).
- Para el día actual se puede comprar hasta las 17:30, una hora y media antes del cierre, aunque el parque todavía no haya abierto.
- Se puede comprar con hasta un mes de anticipación.
- La fecha se ingresa en formato dd/mm/aaaa y debe ser una fecha existente.

- Cada usuario puede comprar hasta 10 entradas para una misma fecha de visita, sumando todas sus compras.
- La edad de cada visitante es un número entero, no negativo.
- Precios: el pase Regular cuesta $10.000 y el VIP $20.000. Los menores de hasta 3 años no pagan. Los visitantes de 4 a 15 años y los de 60 años o más pagan la mitad.
- Además de lo que piden los criterios, se adjunta un archivo junto al mail.

Diseño de la Solución

Se ha utilizado el lenguaje de programación Python con la interfaz Flet. Para las pruebas automatizadas se utiliza pytest.

Se estructura con el modelo BCE:

- Boundaries: todo lo que ve el usuario o sale del sistema: las pantallas Flet, el mail HTML y el PDF de la factura.
- Controllers: coordina el caso de uso “Comprar entrada”. Valida los datos, controla el límite de entradas por fecha, cobra a través de la pasarela, emite la factura, guarda la entrada en el usuario y envía el mail de confirmación.
- Entities: contiene las reglas del negocio, los errores propios y la integración con Mercado Pago.

Se eligió esta estructura porque permite probar las reglas sin levantar la interfaz ya que los tests trabajan sobre las entidades o sobre el controller.

Se ha tomado la decisión de que la fecha y hora actual se recibe como parámetro, al igual que el momento de la compra en lugar de leer el reloj del sistema. Se creó de esta forma ya que las pruebas usan un valor fijo así el resultado del test no depende del día ni de la hora en que se ejecutan. Sin esto, las pruebas de fecha actual y del horario de las 17:30 cambiarían de resultado según cuándo se corran.

Las pruebas contra Mercado Pago y el servidor de mail reales solo corren si se activan con variables de entorno, así que una ejecución normal no cobra ni envía mails.

Pruebas Unitarias

Pruebas de fecha

1.1	Probar que la fecha de visita sea la fecha actual.	Pasa
1.2	Probar que la fecha de visita sea posterior a la fecha actual.	Pasa
1.3	Probar que la fecha de visita sea anterior a la fecha actual.	Falla.
1.4	Probar que la fecha tenga el formato correspondiente.	Pasa
1.5	Probar que la fecha no tenga el formato correspondiente.	Falla
1.6	Probar una fecha correspondiente a un día en el que el parque está abierto.	Pasa
1.7	Probar una fecha correspondiente a un día en que el parque está cerrado.	Falla
1.8	Probar que se cargó la fecha de visita en el formulario.	Pasa
1.9	Probar una fecha anterior a que pase un mes desde la fecha actual.	Pasa
1.10	Probar una fecha posterior a que pase un mes desde la fecha actual.	Falla
1.11	Probar para la fecha actual en un horario anterior a las 17:30.	Pasa
1.12	Probar para la fecha actual en el horario 17:30.	Pasa
1.13	Probar para la fecha actual en un horario posterior a las 17:30.	Falla
1.14	Probar una fecha correspondiente a un domingo.	Pasa
1.15	Probar una fecha correspondiente a un día festivo.	Falla
1.16	Probar la compra para la fecha actual hasta las 17:30, sin exigir que el parque ya haya abierto.	Pasa

Pruebas de cantidad de entradas

2.1	Probar una cantidad de entradas entre 1 y 10.	Pasa
2.2	Probar exactamente 10 entradas.	Pasa
2.3	Probar una cantidad de entradas mayor a 10.	Falla
2.4	Probar una cantidad igual a 0.	Falla
2.5	Probar una cantidad negativa.	Falla
2.6	Probar una cantidad decimal.	Falla
2.7	Probar una cantidad no numérica.	Falla
2.8	Probar que la cantidad de entradas sea la misma cantidad de edad a cargar.	Pasa
2.9	Probar que se cargó la cantidad de entradas de visita en el formulario.	Pasa
2.10	Probar cargar 10 o menos entradas en varias transacciones.	Pasa
2.11	Probar cargar más de 10 entradas en varias transacciones.	Falla

Pruebas de edad

3.1	Probar cargar una edad numérica y entera.	Pasa
3.2	Probar ingresar una edad de un visitante mayor a 100 años.	Falla
3.3	Probar cargar una edad negativa.	Falla
3.4	Probar cargar una edad no numérica.	Falla
3.5	Probar cargar una edad decimal.	Falla
3.6	Probar ingresar correctamente la edad de todos los visitantes.	Pasa

3.7	Probar ingresar la edad de todos los visitantes y que una de ellas sea inválida.	Falla
3.8	Probar no cargar la edad de alguno de los visitantes.	Falla

Pruebas de tipo de pase

4.1	Probar seleccionar un pase VIP.	Pasa
4.2	Probar seleccionar un pase Regular.	Pasa
4.3	Probar un tipo de pase distinto de VIP o Regular.	Falla

Pruebas de forma de pago

5.1	Probar seleccionar pago en efectivo.	Pasa
5.2	Probar seleccionar pago con tarjeta a través de Mercado Pago.	Pasa
5.3	Probar realizar la compra sin seleccionar una forma de pago.	Falla
5.4	Probar una forma de pago distinta de efectivo o tarjeta.	Falla

Pruebas de Mercado Pago

6.1	Probar que al confirmar una compra con tarjeta se redirija a Mercado Pago.	Pasa
6.2	Probar un pago rechazado por falta de saldo y se informa del problema.	Pasa.
6.3	Probar que, si el pago no se confirma, no se envíe el mail de confirmación.	Pasa.

Pruebas de email

7.1	Probar que una compra confirmada envíe el mail de confirmación.	Pasa
7.2	Probar el envío de un mail de confirmación vacío.	Falla

Pruebas de importe

8.1	Probar una entrada VIP con una edad de 3 años o menor y el monto de la entrada es cero.	Pasa
8.2	Probar una entrada Regular con una edad de 3 años o menor y el monto de la entrada es cero.	Pasa
8.3	Probar una entrada VIP con edad menor o igual a 15 años y el monto de la entrada es 10000.	Pasa
8.4	Probar una entrada Regular con edad menor o igual a 15 años y el monto de la entrada es 5000.	Pasa
8.5	Probar una entrada VIP con edad mayor o igual a 60 años y el monto de la entrada es 10000.	Pasa
8.6	Probar una entrada Regular con edad mayor o igual a 60 años y el monto de la entrada es 5000.	Pasa
8.7	Probar la visualización del subtotal y el total antes de confirmar la compra	Pasa
8.8	Probar que al cambiar el tipo de pase se actualicen los importes	Pasa
8.9	Probar que al cambiar la cantidad de entradas se actualicen los importes.	Pasa
8.10	Probar que al cargar una edad inválida se quiten los importes anteriores.	Pasa
8.11	Probar que al cambiar una edad se actualice el total.	Pasa

Finalización de la compra

9.1	Probar que al finalizar una compra correcta se informe la cantidad de entradas compradas y la fecha de visita.	Pasa

Conclusión

Aplicar TDD a la user story “Comprar Entradas” nos obligó a definir cada regla con precisión antes de programarla: el formato de la fecha, el horario límite, la anticipación máxima y los valores inválidos. Así, las pruebas terminaron funcionando como una especificación detallada de la user story.

El ciclo también guió el diseño. Para que las pruebas fueran repetibles e independientes, la fecha actual se recibe como parámetro, y la pasarela de pago y el servicio de mail se inyectan en el controller, lo que nos permitió usar dobles de prueba. Junto con la estructura BCE, esto hizo posible probar casi todas las reglas sin levantar la interfaz.

Las mayores dificultades estuvieron en las dependencias externas (Mercado Pago, el mail y la pantalla de Flet). En síntesis, TDD exigió más trabajo al inicio, pero nos dio un código con reglas explícitas, pruebas que documentan su comportamiento y confianza para refactorizar sin romperlo.